STEP-BY-STEP DEPLOYMENT GUIDE

Dockerized Apache Website on AWS EC2

Build an HTML/CSS site → Package with Docker & Apache → Push to Docker Hub → Deploy publicly on an EC2 instance

Docker | Apache HTTPD | Docker Hub | AWS EC2

What You'll Build



This guide takes you through **9 working steps** — from writing your webpage locally, to containerizing it with Apache, publishing the image to Docker Hub, and finally running it live on an EC2 instance so it's reachable by anyone on the internet.

Contents

Prerequisites	Page 3
Step 1 — Create your HTML/CSS website	Page 3
Step 2 — Write the Dockerfile (Apache)	Page 4
Step 3 — Build the Docker image locally	Page 4
Step 4 — Test the container on localhost	Page 5
Step 5 — Push the image to Docker Hub	Page 5
Step 6 — Launch an EC2 instance	Page 6
Step 7 — Install Docker on EC2	Page 7
Step 8 — Pull the image & run on EC2	Page 8
Step 9 — Open your site to the world	Page 8
Cleanup & Troubleshooting	Page 9

Prerequisites

Requirement	Notes
Docker Desktop	Installed and running on your local machine (Windows/Mac/Linux)
Docker Hub account	Free account at hub.docker.com — this is your image registry
AWS account	With permission to launch EC2 instances
SSH client	Terminal (Mac/Linux) or PuTTY / Windows Terminal (Windows)
Text editor	VS Code or any editor to write HTML/CSS/Dockerfile

1 Create Your HTML/CSS Website

On your local machine, create a project folder and add your website files.

```
mkdir apache-docker-site
cd apache-docker-site
```

Create `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Dockerized Site</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello from Apache inside Docker!</h1>
  <p>This page is served by Apache HTTPD running in a container on AWS EC2.</p>
</body>
</html>
```

Create `style.css` in the same folder:

```
body {
  font-family: Arial, sans-serif;
  background: #f4f7fb;
  text-align: center;
  padding-top: 80px;
}
h1 { color: #0b3d91; }
p { color: #333; font-size: 18px; }
```

Tip
Your folder should now contain: `index.html`, `style.css`, and shortly a `Dockerfile` — all at the same level.

2 Write the Dockerfile (Apache Base Image)

Create a file named exactly `Dockerfile` (no extension) in the project folder:

```
# Use the official Apache HTTP Server image
FROM httpd:2.4

# Copy your website files into Apache's default web root
COPY ./index.html /usr/local/apache2/htdocs/
COPY ./style.css  /usr/local/apache2/htdocs/

# Apache listens on port 80 by default
EXPOSE 80
```

Why httpd:2.4?
This is the official Apache image maintained on Docker Hub. Its default web root inside the container is `/usr/local/apache2/htdocs/` — that's where your site files must land.

3 Build the Docker Image Locally

From inside the project folder, run:

```
docker build -t my-apache-site:1.0 .
```

Verify the image was created:

```
docker images
```

4 Test the Container on Localhost

```
docker run -d -p 8080:80 --name apache-test my-apache-site:1.0
```

Open your browser and visit:

```
http://localhost:8080
```

Working check
You should see "Hello from Apache inside Docker!" styled with your CSS. If it loads, your image is good — stop and remove the test container before moving on:

```
docker stop apache-test
docker rm apache-test
```

5 Push the Image to Docker Hub

Log in to Docker Hub from your terminal:

```
docker login
```

Tag your image with your Docker Hub username (replace `yourusername`):

```
docker tag my-apache-site:1.0 yourusername/my-apache-site:1.0
```

Push it:

```
docker push yourusername/my-apache-site:1.0
```

Confirm
Log in to hub.docker.com in your browser → Repositories. You should see `yourusername/my-apache-site` listed as public (or private) with tag `1.0`.

6 Launch an EC2 Instance

1. Go to the AWS Console → **EC2** → **Launch Instance**.
2. Name it, e.g. `apache-docker-server`.
3. **AMI:** choose *Amazon Linux 2023* (or Ubuntu 22.04 — commands adjust slightly, noted below).
4. **Instance type:** `t2.micro` (free-tier eligible) is enough for a static site.
5. **Key pair:** create a new key pair (e.g. `docker-key`), download the `.pem` file, keep it safe.
6. **Network settings → Security Group:** allow these inbound rules:

Type	Port	Source
SSH	22	My IP
HTTP	80	Anywhere (0.0.0.0/0)

7. Click **Launch Instance** and wait until the instance state is *Running*.
8. Copy the instance's **Public IPv4 address** — you'll need it for SSH and for viewing the site.

Important
Without the HTTP (port 80) rule open to 0.0.0.0/0, your site will run fine on the server but nobody on the internet will be able to reach it.

7 Connect & Install Docker on EC2

Set correct permissions on your key and connect via SSH (replace the IP and key path):

```
chmod 400 docker-key.pem
ssh -i docker-key.pem ec2-user@YOUR_EC2_PUBLIC_IP
```

On Amazon Linux 2023, install and start Docker:

```
sudo yum update -y
sudo yum install -y docker
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

Log out and reconnect (`exit` then SSH again) so the group change applies. Confirm Docker works:

```
docker --version
```

Using Ubuntu instead?

Replace the install block with:

```
sudo apt update -y
sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ubuntu
```

8 Pull the Image & Run It on EC2

Still connected via SSH, pull your image from Docker Hub:

```
docker pull yourusername/my-apache-site:1.0
```

Run the container, mapping the container's port 80 to the host's port 80:

```
docker run -d -p 80:80 --restart unless-stopped --name apache-live yourusername/my-apache-site:1.0
```

Confirm it's running:

```
docker ps
```

--restart unless-stopped

This makes your container automatically restart if the EC2 instance reboots — good practice for a "published" site.

9 Open Your Site to the World

From any browser, anywhere, visit:

```
http://YOUR_EC2_PUBLIC_IP
```

You should see your HTML/CSS page — now served by Apache, running inside Docker, on a public AWS EC2 server, pulled directly from Docker Hub.

Working end-to-end checklist

✓ Site loads on localhost:8080 (Step 4)
✓ Image visible on Docker Hub under your username (Step 5)
✓ EC2 instance state = Running, port 80 open (Step 6)
✓ `docker ps` shows apache-live as Up (Step 8)
✓ http://YOUR_EC2_PUBLIC_IP loads the page in a browser (Step 9)

Cleanup & Troubleshooting

Problem	Fix
Site doesn't load on public IP	Re-check the Security Group has HTTP/80 open to 0.0.0.0/0
`docker: permission denied` on EC2	Make sure you re-logged in after `usermod -aG docker`
Container exits immediately	Run `docker logs apache-live` to see the error
Old version still showing	Rebuild with a new tag, push, then `docker pull` again on EC2 and restart the container

To stop and remove the container on EC2 later:

```
docker stop apache-live
docker rm apache-live
```

To terminate the EC2 instance entirely (stops billing), go to AWS Console → EC2 → Instances → select instance → Instance State → Terminate.